Predicting Customer Campaign Response: A Comparative Study of Supervised Learning, Randomized Optimization, and Unsupervised Feature Engineering

Xiaofeng Zhao
Khoury College of Computer Sciences
Northeastern University
zhao.xiaofe@northeastern.edu

Tyler Sarna
Khoury College of Computer Sciences
Northeastern University
sarna.ty@northeastern.edu

Xiangying Sun
Khoury College of Computer Sciences
Northeastern University
sun.xiangyi@northeastern.edu

I. Objectives and Significance

We want to figure out if a customer will respond to a marketing campaign—it's a yes-or-no question. We're using the Customer Personality Analysis dataset to help us make this prediction. Knowing who will respond helps businesses focus their efforts on the right people and not waste time and money on those who probably won't be interested.

One big problem we're facing is that about 79% of the customers in our data never took up any offer, which means our classes are pretty uneven. If we used a simple classifier that always said "no," it would be right about 79% of the time, but it wouldn't be helpful in real life. So, instead of just looking at accuracy, we're using the F1-score as our main way to measure how well our classifier is doing. This score takes into account both the times we miss a positive result and the times we get a false alarm, which makes it a more useful measure of how well our classifier is really working.

Rather than relying on a single model, we run eight algorithms from three different families on the same preprocessed data:

- **Section IV**: SVM and KNN, two classical approaches we use as baselines.
- **Section V**: Decision Tree, GAM, XGBoost, and EBM—a sequence that trades readability for accuracy.
- **Section VI**: A neural network trained by SGD, then by randomized optimizers (RHC, SA, GA), and finally augmented with unsupervised features from clustering and dimensionality reduction.

Prior work on this type of customer data has used logistic regression [2], random forests [3], and various boosting methods, but most studies examine only one paradigm at a time. We put all

three paradigms on equal footing—same features, same train/test split, same metric—so the results are directly comparable.

II. Background

A. Classical Machine Learning

Support Vector Machines (SVM) are a way to find the best hyperplane that separates two groups of data with the biggest gap between them. When the data can't be separated by a straight line, kernel functions map the inputs into a higher-dimensional space where a linear separator may exist. The really useful thing about SVM is that it only relies on a small subset of the training data—the support vectors—to make its predictions. This means it can handle situations where there are a lot of features but not a lot of samples to work with.

K-Nearest Neighbors (KNN) classifies each query point by a majority vote of its k closest training neighbors. The algorithm stores the entire training set and has no explicit training phase. In high-dimensional spaces, however, all pairwise distances tend to converge, which makes the notion of "nearest" less meaningful and often hurts performance.

B. Decision Tree–Based Models

Decision Trees split the feature space along one feature at a time, choosing the threshold that yields the largest reduction in impurity (e.g., Gini index). The resulting tree is easy to inspect but can overfit badly if not pruned or depth-limited.

Generalized Additive Models (GAM) work by learning a separate smooth curve (spline) for each input feature, then summing these curves to make a prediction. You can plot each curve individually, which means you can see exactly how each feature affects the output. However, GAM cannot handle interactions between features on its own unless you manually add in interaction terms.

XGBoost [9] trains an ensemble of shallow trees one after another; each new tree fits the residual error left by all previous trees. Built-in L_1/L_2 regularization and column subsampling help prevent overfitting.

Explainable Boosting Machines (EBM) [4] take the idea of GAMs and add a boosting procedure: they cycle through features repeatedly, fitting a small tree to each feature's residual in turn, and optionally add pairwise interaction terms. The output is still a sum of per-feature functions, so it remains interpretable, yet the boosted fitting often reaches accuracy close to that of a full ensemble.

C. Neural Networks and Optimization

Neural Networks (NN) stack layers of weighted, nonlinear transformations. We use a feedforward network with ReLU activations and a sigmoid output, trained to minimize Binary Cross-Entropy (BCE). Standard training uses backpropagation to compute gradients and SGD (or a variant) to update weights.

We also experiment with three gradient-free search methods:

- **Random Hill Climbing (RHC)**: perturb the current weight vector slightly and keep the change only if loss decreases.
- **Simulated Annealing (SA)**: like RHC, but sometimes accepts worse solutions; the acceptance probability drops over time as the "temperature" cools.
- **Genetic Algorithm (GA)**: maintains a population of weight vectors that breed, mutate, and compete over many generations.

D. Unsupervised Learning and Dimensionality Reduction

K-Means assigns each point to the nearest of k centroids and recomputes centroids until convergence. **Expectation Maximization (EM)** with

Gaussian Mixtures assigns soft, probabilistic memberships instead of hard labels. **PCA** changes the coordinate system so the first axis captures the most variation in the data, then the second axis captures the next most, and so on. **ICA** looks for linear projections that are as statistically independent as possible, often surfacing signals with heavy tails. **Random Projection (RP)** multiplies the data by a random matrix; the Johnson–Lindenstrauss lemma guarantees that pairwise distances are approximately preserved if enough projected dimensions are kept.

III. Methods

A. Dataset Description

The Customer Personality Analysis dataset [1] from Kaggle has 2,240 records and 29 raw features, covering:

- **Demographics**: Year_Birth, Education, Marital_Status, Income, Kidhome, Teenhome
- **Purchase behavior**: MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds
- **Channel usage**: NumWebPurchases, NumCatalogPurchases, NumStorePurchases, NumWebVisitsMonth
- **Campaign response**: AcceptedCmp1–5, Response

We're looking at a target called `AcceptedAny`, which is a combination of five different campaigns—AcceptedCmp1 through AcceptedCmp5. A customer is considered positive if they accepted any one of these campaigns. The resulting class split is roughly 79% negative to 21% positive.

B. Data Preprocessing

Every experiment uses the same preprocessing:

1) **Missing values**: 24 missing `Income` entries are filled with the column median.
2) **Date conversion**: `Dt_Customer` is turned into `Customer_Since_Months`, counted from a reference date of July 1, 2014.
3) **Age**: $\texttt{Age} = 2015 - \texttt{Year_Birth}$.
4) **Encoding**: Education and Marital_Status are one-hot encoded and then collapsed back into single integer columns with `LabelEncoder`.
5) **Dropped columns**: `ID` (just a row number), `NumDealsPurchases` and `Response` (both have a direct connection to the target and would leak information).
6) **Target**: `AcceptedAny` is computed; the five original campaign columns are removed.

After these steps the dataset has 2,240 rows and about 30 features.

C. Data Splitting and Scaling

We start by using `StandardScaler` to make sure all the features are on the same scale—zero mean and unit variance. Then we split the data into 60% training, 20% validation, and 20% test by stratified sampling (`random_state=42`) so that each subset keeps the same positive-to-negative ratio. Where noted, we also run 5-fold stratified cross-validation.

D. Evaluation Strategy

F1-score is our primary metric because accuracy is misleading on imbalanced data (a constant "no" predictor already scores 79%). We also report accuracy, precision, recall, and wall-clock training time. For clustering quality we look at silhouette score, inertia, and log-likelihood.

IV. Classical Machine Learning

A. SVM Hyperparameter Analysis

We experimented with two different kernels, polynomial and RBF, and tested them with four different values for the regularization parameter C (Table I). The best outcome came from using the RBF kernel with $C = 100$, which gave us an F1

score of 0.564. However, when we increased C to 1000, our F1 score actually decreased, likely because the decision boundary became too closely fitted to the training data. Overall, the RBF kernel outperformed the polynomial kernel at every C value we tested, which isn't surprising since RBF can handle a wider range of decision boundaries.

TABLE I
SVM PERFORMANCE WITH DIFFERENT KERNELS AND C VALUES

Kernel	C	Acc.	F1	Prec.	Rec.
Poly	1	0.825	0.406	0.682	0.290
Poly	10	0.822	0.462	0.616	0.377
Poly	100	0.812	0.518	0.540	0.499
Poly	1000	0.785	0.508	0.486	0.525
RBF	1	0.832	0.463	**0.707**	0.340
RBF	10	**0.840**	0.540	0.663	0.460
RBF	100	0.835	**0.564**	0.605	0.535
RBF	1000	0.811	0.545	0.530	**0.551**

Learning Curve. Fig. 1 plots train and test error at three training set sizes. Both curves are nearly flat, meaning SVM reaches about the same performance whether it sees 400 or 1,344 training samples. This is expected: SVM only needs the support vectors close to the margin, so extra interior points add little.

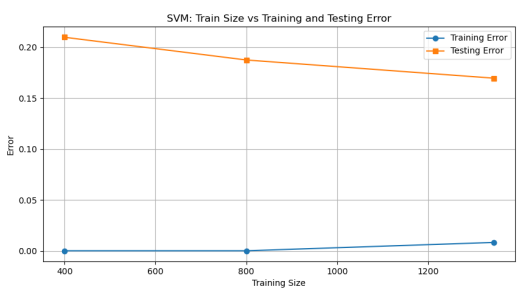

Fig. 1. SVM learning curve. Error barely changes with training size.

B. KNN Hyperparameter Analysis

We tried out different values for K, specifically 3, 5, 7, and 9, using both Euclidean and Manhattan distance (Table II). It turns out that $K = 3$ with Euclidean distance gives the best results, with an F1 score of 0.539. But as K gets bigger, the F1 score drops pretty quickly. This makes sense because when you're dealing with a space that has around 30 dimensions, the idea of a "nearest neighbor" starts to lose meaning—all the points are pretty close together, so adding more neighbors just means you're adding more noise. At smaller values of K, Euclidean distance works a bit better, but as K gets bigger, Manhattan distance is more resilient to the high-dimensional concentration effect.

TABLE II
KNN PERFORMANCE WITH DIFFERENT K AND DISTANCE METRIC

K	Distance	F1-Score
3	Manhattan	0.537
3	Euclidean	**0.539**
5	Manhattan	0.514
5	Euclidean	0.524
7	Manhattan	0.474
7	Euclidean	0.434
9	Manhattan	0.451
9	Euclidean	0.450

Learning Curve. Fig. 2 shows that KNN benefits from more data, but the improvement tapers off fairly quickly. It sits somewhere between SVM (indifferent to data size) and the neural network (hungry for data).

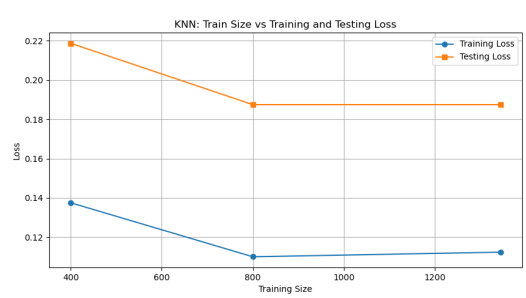

Fig. 2. KNN learning curve.

TABLE III
CLASSICAL MODEL COMPARISON (BEST CONFIGURATIONS)

Model	Best Config	F1	Time (s)
SVM	RBF, C=100	**0.564**	0.64
KNN	K=3, Euclidean	0.539	0.01

C. Classical ML Summary

When it comes to performance, SVM clearly outperforms KNN by about 2.5 points in F1. Both models train in under a second, so the difference in training time is negligible. This gap isn't surprising given our dataset: with around 30 features and only about 2,200 samples, the kernel trick gives SVM a distinct advantage over KNN, which relies solely on distance-based methods.

V. MODELS ON AN INTERPRET-ABILITY SPECTRUM

1) Decision Tree: [Decision Tree HyperParameter Analysis]

Table IV presents Decision Tree results across various depths. We see improvement up to the depth of 7, and past that we see declines in the F1 score. This is likely to be cause by overfitting and the lack of regularization in the model's methodology. Greater depth simply causes the model to generalize poorly to new data. Our optimized depth results in an Accuracy of 0.85, an F1-Score of 0.59, a Precision of 0.67, and a Recall of 0.53.

TABLE IV
DT PERFORMANCE WITH DIFFERENT DEPTH VALUES

Depth	F1
5	0.4719
7	0.5293
10	0.5194
15	0.5141
20	0.5178

Learning Curve. Fig. 3 shows that the Decision tree is benefiting from increasing amounts of data even though the increase isn't smooth. It is possible that further increases in the amount of data could push performance higher as the tree continues to capture the patterns in the data more accurately.

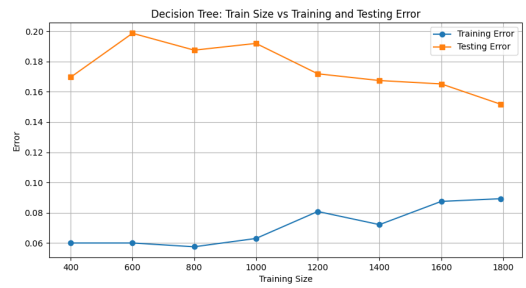

Fig. 3. Marketing: Learning Curve (DT). Performance is increasing on average as training size is increasing.

[DT Interpretability] **Interpretability level by level.** Fig. 4 shows that the Decision tree can be audited by us at every level. If we have any questions or concerns about model performance, the user can check exactly why the model made the decision it did and further we can know which features are contributing the most to the outcomes.

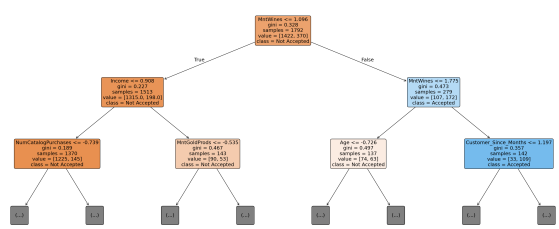

Fig. 4. Marketing: Interpretability (DT). We can check what caused the decision at each step of the decision made by a DT

Feature Importance Fig. 5 shows us the importance of the top ten features of the model. It also could give us clear insight on what characteristics might be most important to consider when planning another marketing campaign.

2) Generalized Additive Model: [GAM Hyper-Parameter Analysis]

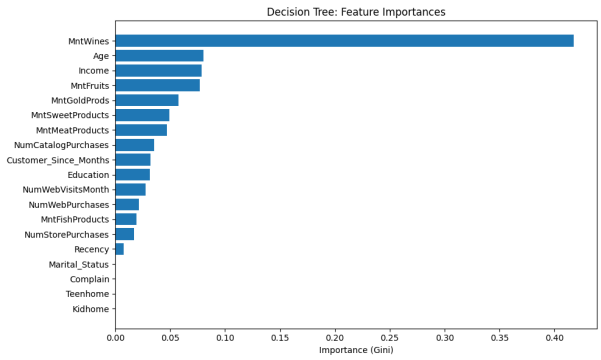

Fig. 5. Marketing: Interpretability (DT). We can check what the most important features are to the model.

Table V presents Generalized Additive model results across various Spline amounts and Lambda values for smoothing. With this, we found the optimal result to be at 45 splines and a lambda value of 150. This seems to be the optimal point where the high splines capture the complexities in the data set while the lambda value is enough to prevent overfitting due to its benefits in smoothing. At 50 splines we see a decrease in F1 scores which is likely due to overfitting and with lower spline values the optimal lambda is lower so the functions are not smoothed as much.

TABLE V
GAM PERFORMANCE WITH DIFFERENT SPLINE AND LAMBDA VALUES

Splines	Lambda	F1
30	80.0	0.5298
35	80.0	0.5419
40	130.0	0.5419
45	150.0	0.5605
50	150.0	0.5570

The learning curve. Fig. 6 shows that the Generalized Additive Model seems to stop benefiting from additional data around 1400 data points. Prior to that there are big improvements and wild swings, but the GAM seems to have extracted all the information it is capable of gathering from the data at this point.

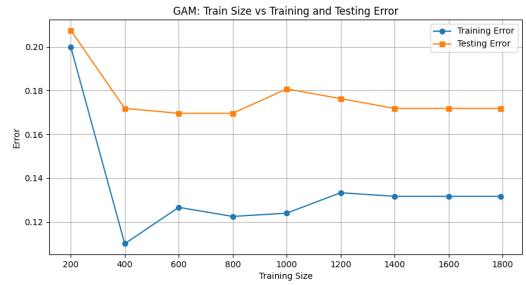

Fig. 6. Marketing: Learning Curve (GAM). Performance increases up to a point

The learning curve. Fig. 7 reflects the belief that the GAM stops improving after 1400 data points. The F1 score stabilizes and does not increase or decrease past this point. This is a clear sign that as long as the patterns remain stable, more data alone will not be able to change the performance of this model.

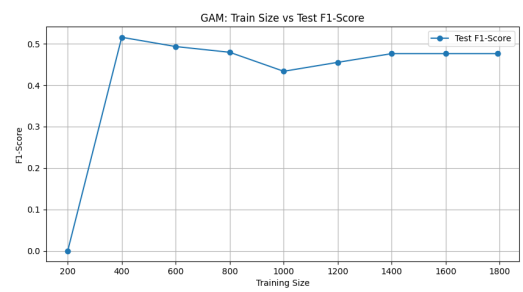

Fig. 7. Marketing: Learning Curve (GAM). Performance is increasing on average as training size is increasing.

[GAM Interpret-ability] **Interpret-ability by Feature.** Fig. 8 and Fig. 9 show how we can look at interpreting the GAM. We can look into exactly what each feature of any data point contributes to the final decision. This allows keen insight into exactly what is going on under the hood and why certain decisions are being made.

3) Explainable Boosting Machine: Boosting and Adding Complexity to GAM: [EBM HyperParameter Analysis]

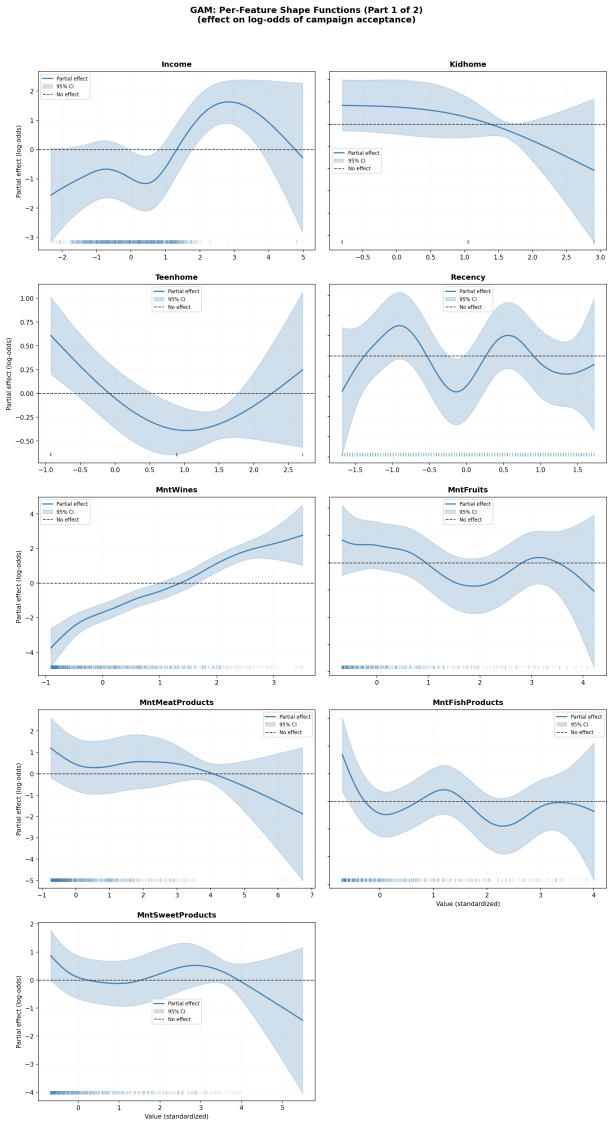

Fig. 8. Marketing: Interpret-ability (GAM). We can audit what each feature is contributing to the decision.

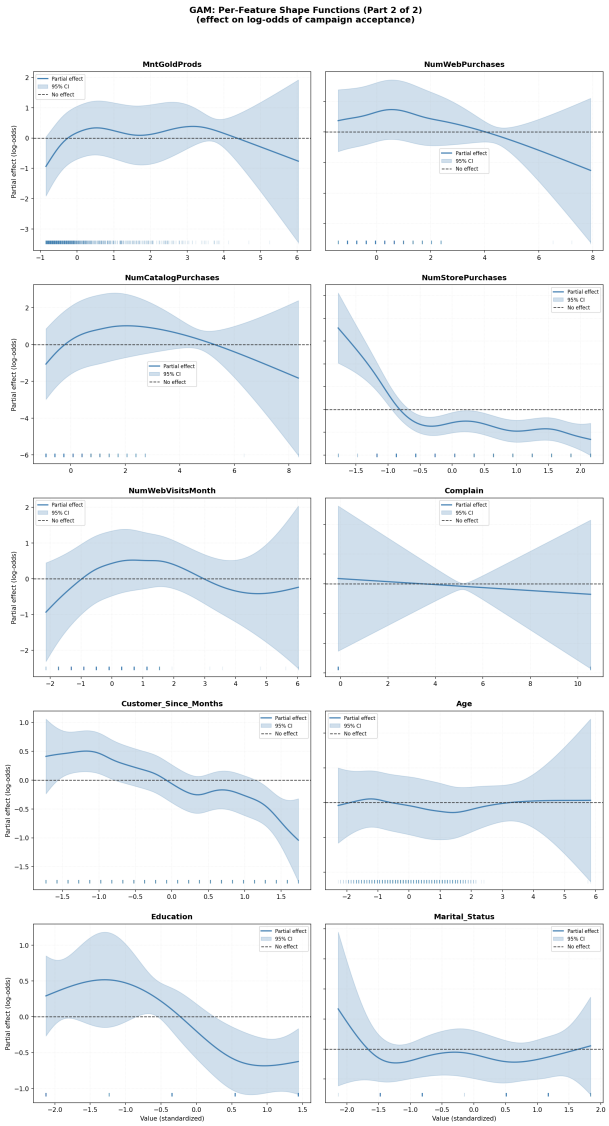

Fig. 9. Marketing: Interpret-ability (GAM). We can audit what each feature is contributing to the decision.

Table VI presents Explainable Boosted Model results with different amounts of Bins and pairwise interactions between features. Both of these allow for huge improvements in performance. In this case, optimal performance is with 48 bins for boosting and 40 pairwise interactions between features. More than this in either category leads to overfitting.

The learning curve. Fig. 10 shows that the Explainable Boosting Model has not plateaued at the current amount of data. This could indicate that the model performance could still improve as error is decreasing as we add more data.

The F1 curve. Fig. 11 reflects the belief that the EBM may not be done improving if we could acquire more data for it. We see a huge peak at our maximum amount of data. This could simply be the model's peak, but it could represent the idea that if more data could be fed to the model that it could continue to improve on the current result.

[EBM Interpret-ability] **Interpret-ability by**

TABLE VI
EBM PERFORMANCE WITH DIFFERENT BINS AND INTERACTIONS VALUES

Bins	Interactions	F1
32	30	0.6001
48	30	0.5836
64	30	0.5969
80	30	0.5873
32	40	0.5994
48	40	0.6027
64	40	0.5907
80	40	0.5928
32	50	0.5894
48	50	0.5991
64	50	0.5933
80	50	0.5907

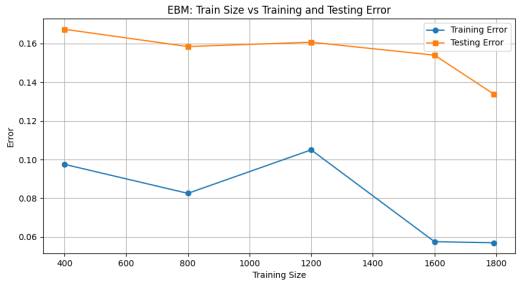

Fig. 10. Marketing: Learning Curve (EBM). Performance increasing as data added

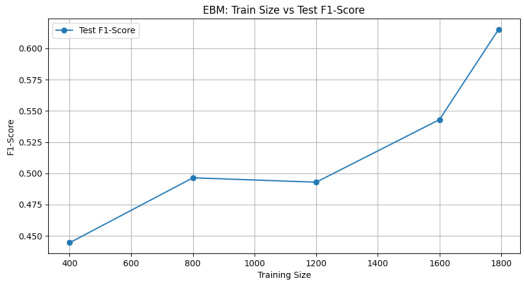

Fig. 11. Marketing: Learning Curve (EBM). Performance is increasing on average as training size is increasing.

Feature. Fig. 12 and Fig. 13 show how we can begin interpreting the EBM. We can look into exactly what each feature of any data point contributes to the final decision. However, that is only the beginning because we also have to look into the 40 additional pairwise interactions between features to get the true picture of how exactly any individual decision was made. This process is still fundamentally transparent, but the boosting and interactions do add an increased level of challenge to interpretability in exchange for improved performance.

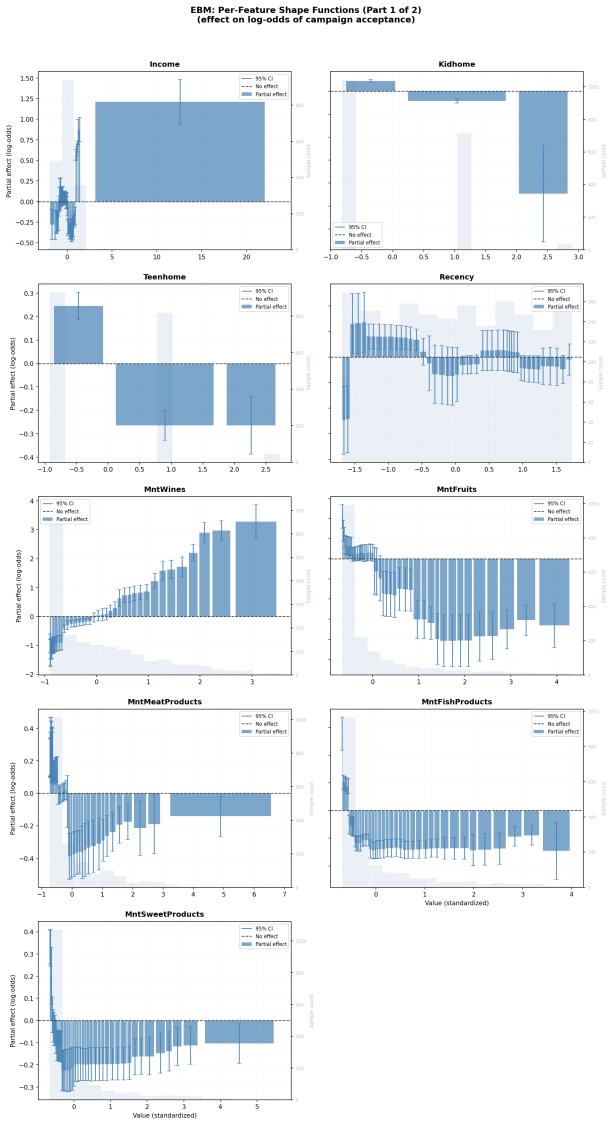

Fig. 12. Marketing: Interpret-ability (EBM). We can audit what each feature is contributing to the decision.

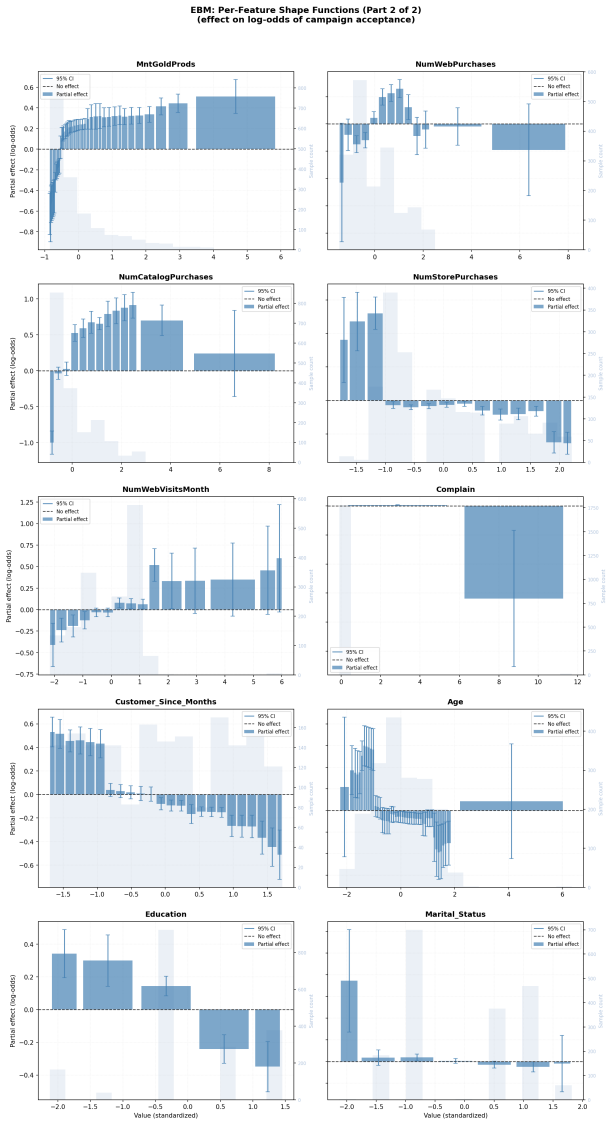

Fig. 13. Marketing: Interpret-ability (EBM). We can audit what each feature is contributing to the decision.

4) XGBoosted Decision Trees: [XGBoost Hyper-Parameter Analysis]

Table VII presents XGBoosted results with different amounts of estimators and max depths. Max depth settles at an ideal result of 7, and the number of estimators settles at 400. Both of these end up being optimal for learning the patterns without overfitting on noise.

The learning curve. Fig. 14 shows that the XGBoosted model seems to have possibly plateaued with the current dataset. In fact, performance dropped slightly as we went from 1600 to 1792 training size.

TABLE VII
XGB PERFORMANCE WITH DIFFERENT ESTIMATOR AND DEPTH VALUES

Estimators	Max-Depth	F1
200	5	0.6044
300	5	0.6112
400	5	0.6088
500	5	0.6138
600	5	0.6209
200	7	0.6276
300	7	0.6347
400	7	0.6383
500	7	0.6322
600	7	0.6289
200	9	0.6175
300	9	0.6210
400	9	0.6200
500	9	0.6298
600	9	0.6264

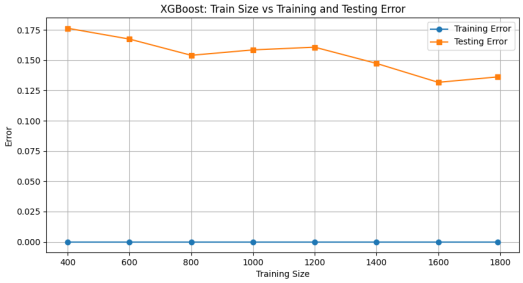

Fig. 14. Marketing: Learning Curve (XGB). Performance increasing to plateau

The F1 curve. Fig. 15 reflects the belief that the XGBoosted model may have plateaued with the current amount of data. The F1 score seems to have possibly topped out with our final training set, but that is no guarantee that more data wouldn't improve performance for this model.

[XGBoost interpretability] **Fundamentally Black-Boxed.** In exchange for the increased performance that XGBoost offers us, it also has a tradeoff of no fundamental methods of

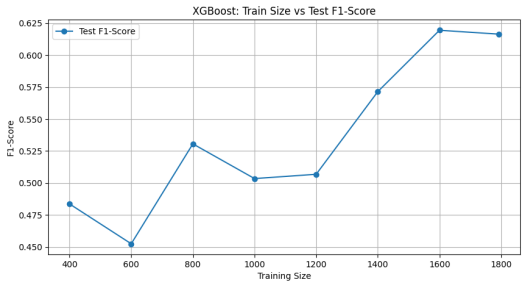

Fig. 15. Marketing: Learning Curve (XGB). Performance is increasing on average as training size is increasing up to a point

interpretability built into it. There are post-hoc methods we could use to derive insights from what the model is doing, but the fundamental logic of the model no longer is interpretable for human purposes.

A. Interpretable and Boosted Models Summary

TABLE VIII
MODEL COMPARISON (BEST CONFIGURATIONS)

Model	Best Config	F1	Time (s)
DT	Depth=7	0.59	0.0732
GAM	Splines=45 Lambda=150	0.48	56.8306
EBM	Interactions=40, Bins=48	0.59	46.4499
XGB	Depth=7 Estimators=400	0.62	1.9419

When it comes to performance, XGBoosting is the clear winner, but it also totally lacks interpretability which makes it completely opaque to human understanding and may be suboptimal in terms of helping a marketing executive actually understand the data and plan a future marketing campaign based on this one. If that is the goal, it may be better to use an EBM or decision tree. The decision tree has the same performance as the EBM for significantly lower training time and cost; however, the EBM might be better for actually explicitly understanding the relationships between the features and it offers the option for experts to adjust relationships without full retraining. The GAM does not appear to offer any advantages when compared to the other models, this data set seems to heavily rely on interactions between the features, and all the other models are capable of capturing those interactions while the GAM cannot.

VI. NEURAL NETWORK–BASED APPROACHES

A. Neural Network Hyperparameter Analysis

Our network is made up of two hidden layers, each with 32 neurons that use the ReLU activation function. The output layer uses a sigmoid function. We trained the network using SGD with a learning rate of 0.001, momentum of 0.9, and weight decay of 1e-4 for 300 epochs using BCE loss.

1) Batch Size: We tried out several batch sizes. With 64, the loss curve goes down smoothly, but with other sizes it either oscillates or gets stuck. We're working with a dataset that has around 1,300 training samples, and with a batch size of 64 we get about 20 gradient updates per epoch. This is a good balance—small enough to add some useful noise, but large enough to keep the variance under control.

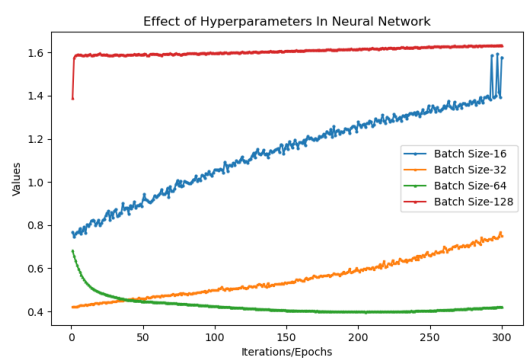

Fig. 16. Batch size vs. test loss. Only batch=64 gives a smooth curve.

2) Learning Rate: Fig. 17 compares learning rates of 0.01, 0.001, and 0.0001. At 0.01 the loss oscillates; at 0.0001 it barely moves in 300 epochs; 0.001 converges without drama.

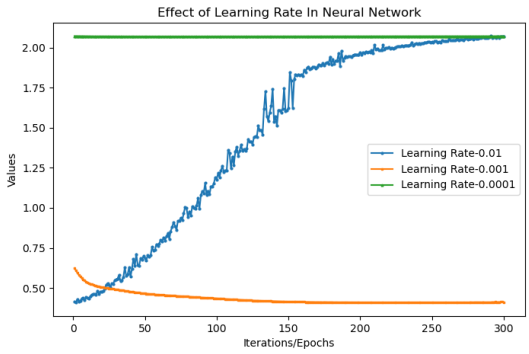

Fig. 17. Learning rate vs. test loss. LR=0.001 works best.

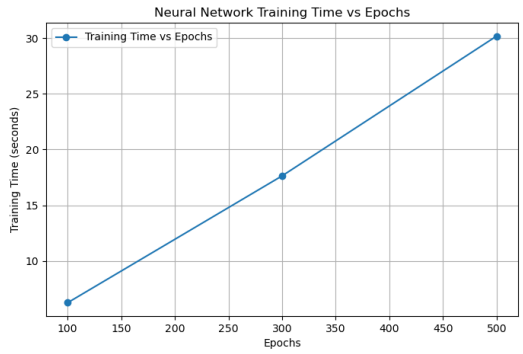

Fig. 19. Training time vs. epochs (neural network).

3) Training Convergence: Fig. 18 shows how the loss changed over 300 epochs for the training, validation, and test sets. Around epoch 230, the three lines start to converge. But then something happens—the training loss keeps going down while the validation loss starts to rise. This is a classic case of overfitting: the model is getting too good at fitting the training data but losing its ability to generalize. In practice, we save the model with the lowest validation loss and stop training there.

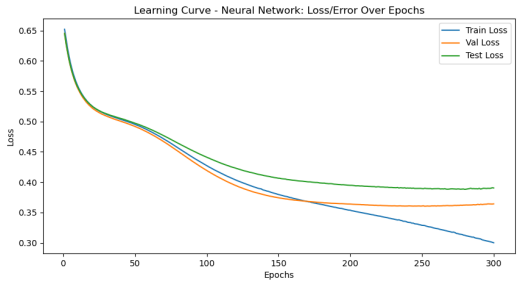

Fig. 18. Loss curves over 300 epochs. Overfitting sets in around epoch 230.

4) Training Time: Fig. 19 plots wall-clock time against number of epochs. The relationship is almost exactly linear; each epoch costs roughly the same amount of computation.

5) Effect of Training Data Size: Fig. 20 makes the data-hunger of neural networks pretty concrete. With only 400 samples the network overfits almost immediately; at 800 samples it overfits by epoch 20; only with the full 1,344 training samples does it converge properly. This really helps explain why neural networks don't do as well as tree-based methods when we're working with small datasets.

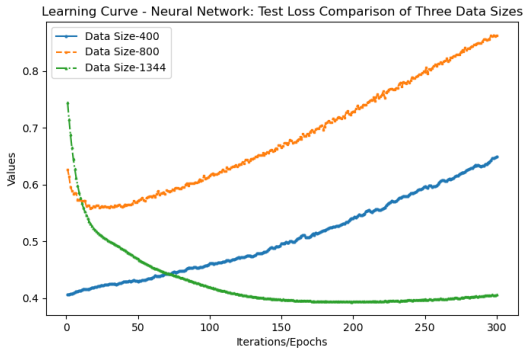

Fig. 20. Dataset size vs. test loss (neural network).

6) Cross-Validation: When we use 5-fold stratified cross-validation, we see the F1 score go from 0.51 to 0.56 (Table IX). However, this improvement comes at a cost—it takes around four times longer to train. The reason it works better is that each fold gives the network a chance to learn from more unique examples, which is really helpful when we're working with a small dataset.

B. Randomized Optimization for NN Weight Training

We use the same network setup, but instead of using gradients we try to find the best weights with

TABLE IX
NEURAL NETWORK: ORDINARY SPLIT VS. CROSS-VALIDATION

Metric	Split	CV (Avg.)
Accuracy	0.83	0.84
F1-Score	0.51	0.56
Precision	0.64	0.67
Recall	0.42	0.48
Time (s)	17.64	73.16

three search algorithms from the `mlrose-hiive` library. All network parameters are flattened into one long vector, and the fitness function evaluates negative BCE loss on the training set (higher is better, since `mlrose` maximizes).

1) Configuration:

- **RHC**: max_iters=300, max_attempts=100.
- **SA**: geometric decay (init_temp=1.0, decay=0.95, min_temp=0.001).
- **GA**: population 200, mutation probability 0.1.

TABLE X
RANDOMIZED OPTIMIZATION: FITNESS AND TIME

Algorithm	Fitness	Time (s)
SGD (baseline)	-0.2916^*	17.64
RHC	-8.7439	0.42
SA	-9.5862	0.59
GA	-1.6517	97.45

*Neg. train loss.

2) Results: RHC and SA finish quickly but end up with really bad fitness values—they get stuck in a certain area of the weight space, which has thousands of dimensions. GA does a lot better because it uses a population of 200 individuals to search more broadly. However, even GA doesn't match SGD, which is not only better at finding a good fit but also a lot faster. The takeaway is straightforward: if you have access to gradients and your loss surface is smooth, there's no good reason to use a gradient-free method.

C. Unsupervised Learning and Feature Engineering

1) Clustering on Original Features: When choosing the right value for k, we calculated silhouette scores for a range from $k = 3$ to $k = 10$ (Fig. 21). K-Means worked best at $k = 5$, while for EM the optimal value was $k = 4$ (from the inertia elbow). Five clusters makes a lot of sense for customer segmentation—you can think of them as different tiers ranging from pretty inactive all the way to high-value.

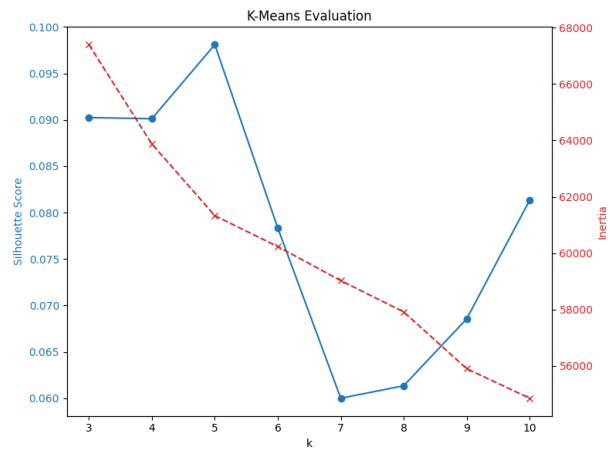

Fig. 21. Silhouette score vs. number of clusters.

K-Means Results. Fig. 22 projects the five clusters onto the first two principal components. The groups separate reasonably well. Looking at income (Fig. 23), each cluster occupies a different income band, which makes intuitive sense as a customer segmentation axis.

EM Results. Fig. 24 and Fig. 25 show the EM counterpart. The cluster boundaries are fuzzier because EM uses soft assignments and full covariance matrices, which lets it model elongated or overlapping groups that K-Means cannot.

2) Dimensionality Reduction Analysis: **PCA.** Fourteen components account for more than 70% of the total variance (Fig. 26). The eigenvalue plot (Fig. 27) shows a steep drop after the first

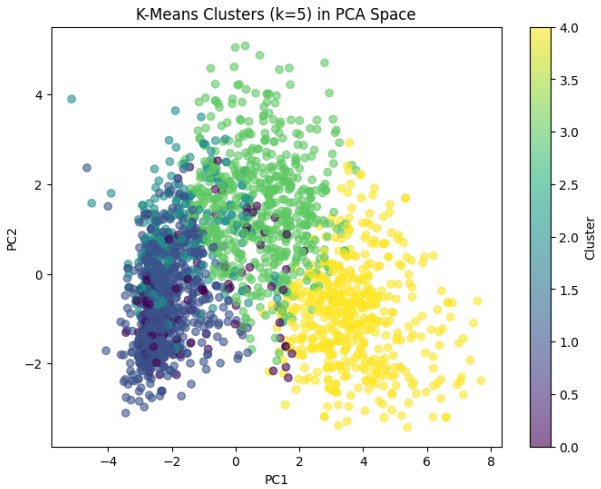

Fig. 22. K-Means clusters in 2D (PCA projection).

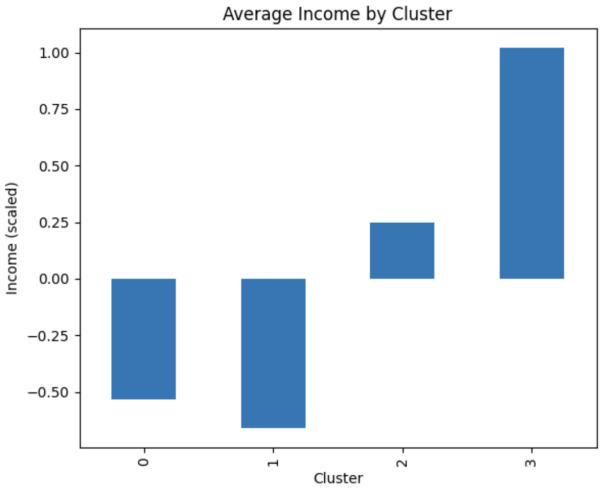

Fig. 23. K-Means clusters by income.

component, then a gradual tail. The data matrix has rank 34 out of 38 columns (Fig. 30), so four features are linearly redundant.

ICA. We extract 10 independent components and measure their kurtosis (Fig. 28). One component stands out with kurtosis around 15—a very peaked, heavy-tailed distribution that likely captures a small group of extreme spenders or otherwise unusual customers. The remaining components have moderate kurtosis.

Random Projection. We project to 10 dimen-

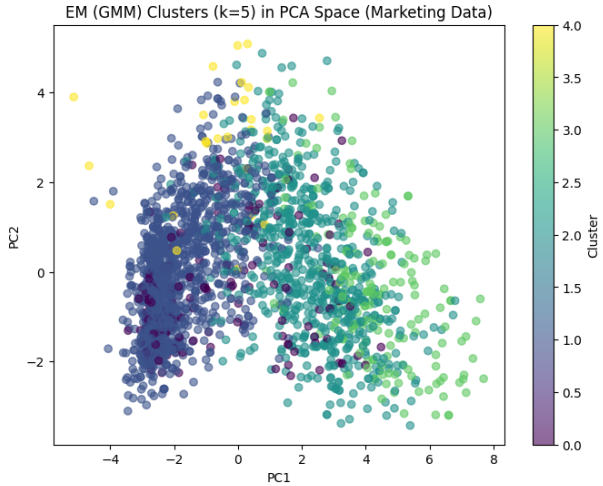

Fig. 24. EM clusters in 2D (PCA projection).

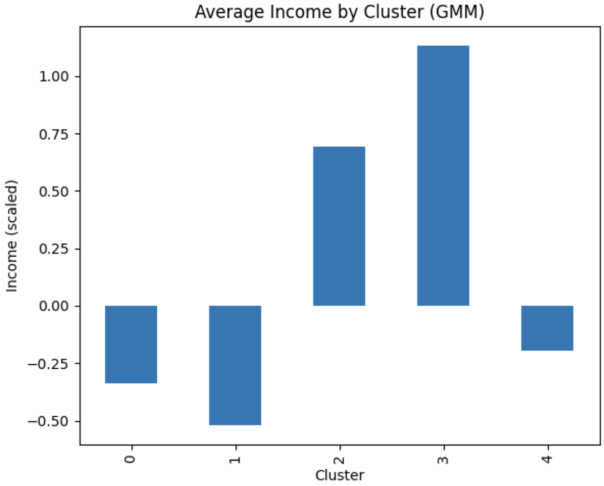

Fig. 25. EM clusters by income.

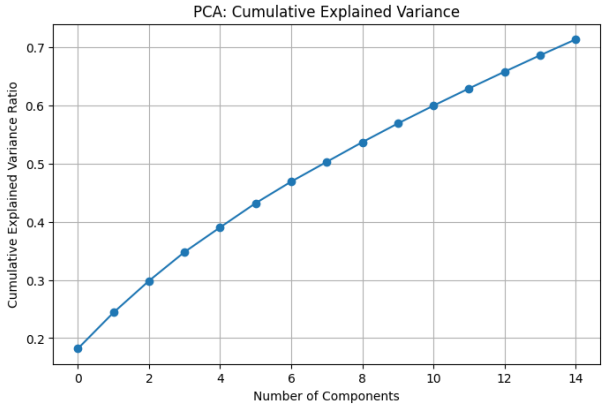

Fig. 26. PCA cumulative explained variance.

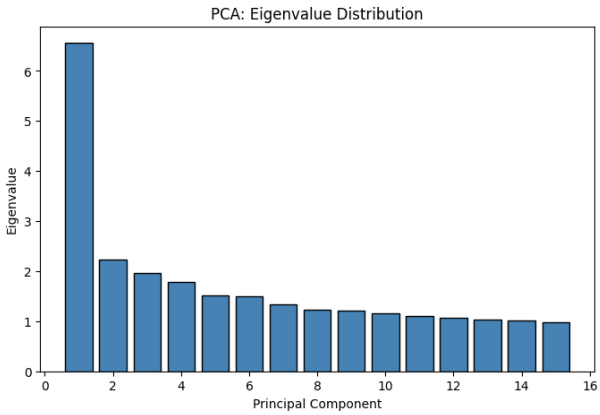

Fig. 27. PCA eigenvalues by component.

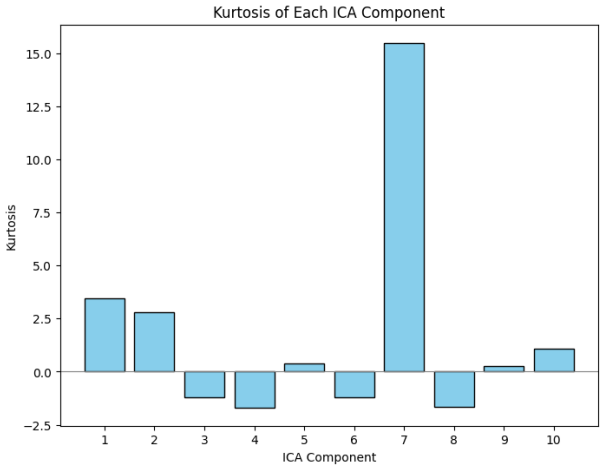

Fig. 28. Kurtosis of 10 ICA components.

sions and measure the Frobenius-norm reconstruction error over 10 different random seeds (Fig. 29). The error ranges from about 237 to 249—a spread of less than 5%—so the projection is fairly stable despite being random.

3) Clustering on Reduced-Dimension Data: ICA is clearly the top performer here, with the best results across the board (Table XI): highest silhouette and lowest inertia with K-Means, and best log-likelihood with EM. This makes sense because ICA is designed to extract non-Gaussian, statistically independent signals—exactly the kind of structure that clustering algorithms are best at identifying. RP doesn't do as well, and PCA falls somewhere

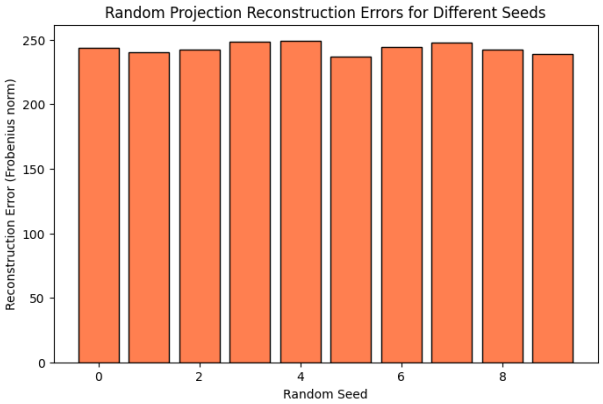

Fig. 29. RP reconstruction error across 10 random seeds.

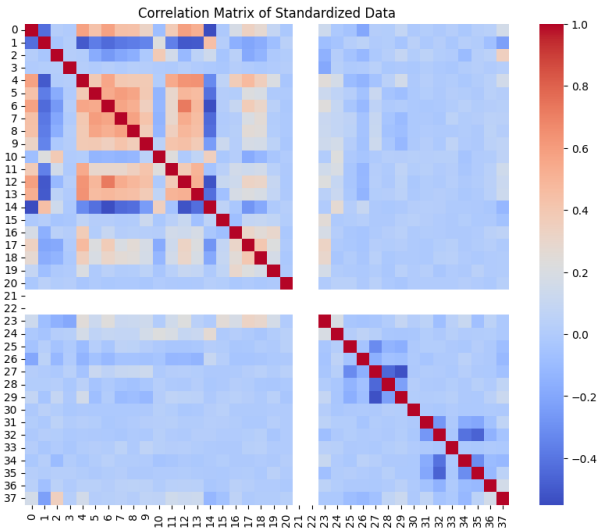

Fig. 30. Data rank (34 of 38 features are linearly independent).

TABLE XI
DIMENSIONALITY REDUCTION + CLUSTERING EVALUATION

DR	Algo	Silh.	Inertia	LogLik.
PCA	KM	0.156	27542	–
PCA	EM	0.055	–	−12.11
ICA	KM	**0.180**	**15381**	–
ICA	EM	0.133	–	**−8.86**
RP	KM	0.122	48115	–
RP	EM	0.033	–	−16.34

in between.

TABLE XII
NN LOSS WITH DIMENSIONALITY REDUCTION

DR	Train	Val	Test	Time (s)
None	0.286	0.365	0.417	36.81
PCA	0.391	0.435	0.443	22.21
ICA	0.381	0.428	0.449	21.10
RP	0.366	0.406	**0.415**	**19.94**

4) Neural Network with Dimensionality Reduction: When you cut the number of features from around 30 to 10, it really speeds up training—we're talking about a 40–46% reduction (Table XII). RP is the fastest and barely affects the test loss (0.415 vs. 0.417 baseline). PCA and ICA aren't as good because they get rid of some features that might be useful for classification even if they don't seem that important at first glance.

TABLE XIII
NN LOSS WITH CLUSTER FEATURES

Method	Train	Val	Test	Time (s)
Baseline	0.286	0.365	0.417	36.81
KM (5 clust.)	0.293	0.345	**0.400**	28.17
EM (4 clust.)	0.263	0.376	0.416	20.39

5) Neural Network with Cluster Features: Adding one-hot-encoded K-Means labels as extra features lowers test loss from 0.417 to 0.400 (about 4%) and cuts training time by 24% (Table XIII). These cluster IDs give the network a pre-computed "which customer tier is this?" feature, saving it from having to learn that grouping from scratch. EM cluster features barely help test loss (0.416); the much lower training loss (0.263) with EM suggests the model is basically memorizing the soft cluster probabilities instead of learning a more general signal.

TABLE XIV
OVERALL MODEL COMPARISON ACROSS ALL SECTIONS

Model	Section	F1	Time (s)
KNN (K=3, Euclid.)	Classical	0.539	0.01
SVM (RBF, C=100)	Classical	0.564	0.64
NN (SGD, 300 ep.)	Neural Net	0.515	17.64
NN (5-fold CV)	Neural Net	0.560	73.16
Decision Tree	Interpret-ability	0.59	0.0732
GAM	Interpret-ability	0.48	56.8306
XGBoost	Interpret-ability	0.62	1.9419
EBM	Interpret-ability	0.59	46.4499

VII. CONCLUSIONS

A. Overall Model Comparison

B. Key Findings

1) **Decision Trees** Decision trees proved to be a remarkably simple and effective model for dealing with this dataset. They are thoroughly interpret-able and provided solid performance despite their relatively low complexity and training cost.
2) **SVM with an RBF kernel is the strongest classical baseline.** F1=0.564, ahead of KNN (0.539) and the single-split NN (0.515). The margin-based formulation works well here because only a few support vectors matter, so the small sample size is less of a problem.
3) **Neural networks don't have enough data to shine.** With 1,344 samples and around 3,000 parameters, the network overfits easily. Cross-validation helps (F1 goes from 0.51 to 0.56) but it's not enough to catch up with tree-based methods. More data would probably make a big difference.
4) **Gradient-free optimization is not competitive for NN training.** SGD reaches a loss of 0.29 in 18 s; the best randomized method (GA) only gets to 1.65 in 97 s. Without gradient information, the search space is simply too large to navigate efficiently.

5) **Cluster-based features give the NN a modest boost.** Adding K-Means labels cuts test loss by 4% and training time by 24%. Among dimensionality reduction methods, ICA produces the best clusters but RP gives the best speed–accuracy trade-off when fed directly to the NN.
6) **GAM's additive assumption is too restrictive.** The GAM does not have any way to model interactions between features in the data, and this restriction meant that the model type fundamentally failed to produce good results for this data set. It also took more time to train than the simpler and better performing decision tree.

What worked: Boosted ensembles (XGBoost, EBM) gave the best predictions. SVM was a solid classical baseline. K-Means cluster labels were a simple but useful form of feature engineering for the neural network.

What did not work: RHC and SA got stuck in bad local optima when training NN weights. GAM's additive form was too limiting. The neural network couldn't keep up with tree-based methods with only 2,240 samples.

Possible next steps:

- Try SMOTE or similar oversampling to address the 79/21 imbalance directly.
- Run feature selection (mutual information, recursive elimination) to cut irrelevant dimensions before modeling.
- If more data become available, try larger or regularized neural networks (dropout, batch normalization).
- Use EBM shape functions together with clustering results to build actionable customer segments for the marketing team.

VIII. Individual Tasks

Xiaofeng Zhao: Designed the project workflow and wrote the preprocessing pipeline. Ran all SVM, KNN, and neural network experiments (Section IV and Section VI), including the randomized optimization and unsupervised feature engineering parts. Produced figures and wrote most of the report.

Tyler Sarna: Built and tuned the Decision Tree, GAM, XGBoost, and EBM models (Section **??**). Carried out the grid searches, generated feature importance plots and shape function visualizations, and wrote the corresponding report sections.

Xiangying Sun: Built and tuned the Neural networks. Carried out dimensionality reduction on neural network. Produced figures and slides for presentation.

We coordinated through shared Jupyter notebooks and weekly discussions. Decisions like the choice of F1 over accuracy and the preprocessing steps were made together.

References

[1] I. Makash, "Customer Personality Analysis," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis/data. [Accessed: Mar. 22, 2025].

[2] P. Kotler and K. L. Keller, *Marketing Management*, 15th ed. Pearson, 2016.

[3] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[4] H. Nori, S. Jenkins, P. Koch, and R. Caruana, "InterpretML: A unified framework for machine learning interpretability," *arXiv preprint arXiv:1909.09223*, 2019.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[7] G. Hayes, "mlrose: Machine Learning, Randomized Optimization and SEarch library for Python," 2019. [Online]. Available: https://github.com/gkhayes/mlrose.

[8] W. B. Johnson and J. Lindenstrauss, “Extensions of Lipschitz mappings into a Hilbert space,” in *Contemporary Mathematics*, vol. 26, pp. 189–206, 1984.

[9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

[10] D. Servén and C. Brummitt, “pyGAM: Generalized additive models in Python,” Zenodo, 2018. [Online]. Available: https://doi.org/10.5281/zenodo.1208723.